

Breadth First Search (BFS): Level Order Traversal

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
    // Function to return Breadth First Traversal of given graph.
```

```
    vector<int> bfsOfGraph(int V, vector<int> adj[]) {
```

```
        int vis[V] = {0};
```

```
        vis[0] = 1;
```

```
        queue<int> q;
```

```
        // push the initial starting node
```

```
        q.push(0);
```

```
        vector<int> bfs;
```

```
        // iterate till the queue is empty
```

```
        while(!q.empty()) {
```

```
            // get the topmost element in the queue
```

```
            int node = q.front();
```

```
            q.pop();
```

```
            bfs.push_back(node);
```

```
            // traverse for all its neighbours
```

```
            for(auto it : adj[node]) {
```

```
                // if the neighbour has previously not been visited,
```

```
                // store in Q and mark as visited
```

```
                if(!vis[it]) {
```

```
                    vis[it] = 1;
```

```
                    q.push(it);
```

```
                }
```

```
            }
```

```
        }  
        return bfs;
```

```
    }
```

```
};
```

```
void addEdge(vector <int> adj[], int u, int v) {
```

```
    adj[u].push_back(v);
```

```
    adj[v].push_back(u);
```

```
}
```

```
void printAns(vector <int> &ans) {
```

```
    for (int i = 0; i < ans.size(); i++) {
```

```
        cout << ans[i] << " ";
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    vector <int> adj[6];
```

```
    addEdge(adj, 0, 1);
```

```
    addEdge(adj, 1, 2);
```

```
    addEdge(adj, 1, 3);
```

```
    addEdge(adj, 0, 4);
```

```
    Solution obj;
```

```
    vector <int> ans = obj.bfsOfGraph(5, adj);
```

```
    printAns(ans);
```

```
    return 0;
```

```
}
```

Depth First Search (DFS)

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

class Solution {
private:
    void dfs(int node, vector<int> adj[], int vis[], vector<int> &ls) {
        vis[node] = 1;
        ls.push_back(node);
        // traverse all its neighbours
        for(auto it : adj[node]) {
            // if the neighbour is not visited
            if(!vis[it]) {
                dfs(it, adj, vis, ls);
            }
        }
    }
public:
    // Function to return a list containing the DFS traversal of the graph.
    vector<int> dfsOfGraph(int V, vector<int> adj[]) {
        int vis[V] = {0};
        int start = 0;
        // create a list to store dfs
        vector<int> ls;
        // call dfs for starting node
        dfs(start, adj, vis, ls);
        return ls;
    }
};

```

```

void addEdge(vector <int> adj[], int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

```

```

void printAns(vector <int> &ans) {
    for (int i = 0; i < ans.size(); i++) {
        cout << ans[i] << " ";
    }
}

```

```

int main()
{
    vector <int> adj[5];

    addEdge(adj, 0, 2);
    addEdge(adj, 2, 4);
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 3);

    Solution obj;
    vector <int> ans = obj.dfsOfGraph(5, adj);
    printAns(ans);

    return 0;
}

```

Number of Provinces

Problem Statement: Given an undirected graph with V vertices. We say two vertices u and v belong to a single province if there is a path from u to v or v to u. Your task is to find the number of provinces.

Pre-req: Connected Components, Graph traversal techniques

Examples:

Input:

Output: 3

```

#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    // dfs traversal function
    void dfs(int node, vector<int> adjLs[], int vis[]) {
        // mark the more as visited
        vis[node] = 1;
        for(auto it: adjLs[node]) {
            if(!vis[it]) {
                dfs(it, adjLs, vis);
            }
        }
    }
public:
    int numProvinces(vector<vector<int>> adj, int V) {
        vector<int> adjLs[V];

        // to change adjacency matrix to list
        for(int i = 0; i < V; i++) {
            for(int j = 0; j < V; j++) {
                // self nodes are not considered
                if(adj[i][j] == 1 && i != j) {
                    adjLs[i].push_back(j);
                    adjLs[j].push_back(i);
                }
            }
        }
        int vis[V] = {0};
        int cnt = 0;
        for(int i = 0; i < V; i++) {
            // if the node is not visited
            if(!vis[i]) {
                // counter to count the number of provinces
                cnt++;
                dfs(i, adjLs, vis);
            }
        }
        return cnt;
    }
};

```

```

int main() {
    vector<vector<int>> adj
    {
        {1, 0, 1},
        {0, 1, 0},
        {1, 0, 1}
    };

    Solution ob;
    cout << ob.numProvinces(adj,3) << endl;

    return 0;
}

```

Rotten Oranges : Min time to rot all oranges : BFS

Problem Statement: You will be given an $m \times n$ grid, where each cell has the following values :

2 - represents a rotten orange

1 - represents a Fresh orange

0 - represents an Empty Cell

Every minute, if a Fresh Orange is adjacent to a Rotten Orange in 4-direction (upward, downwards, right, and left) it becomes Rotten.

Return the minimum number of minutes required such that none of the cells has a Fresh Orange. If it's not possible, return -1.

Examples:

Example 1:

Input: grid - [[2,1,1] , [0,1,1] , [1,0,1]]

Output: -1

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int orangesRotting(vector<vector<int>>& grid) {
```

```
    if(grid.empty()) return 0;
```

```
    int m = grid.size(), n = grid[0].size(), days = 0, tot = 0, cnt = 0;
```

```
    queue<pair<int, int>> rotten;
```

```
    for(int i = 0; i < m; ++i){
```

```
        for(int j = 0; j < n; ++j){
```

```
            if(grid[i][j] != 0) tot++;
```

```
            if(grid[i][j] == 2) rotten.push({i, j});
```

```
        }
```

```
    }
```

```
    int dx[4] = {0, 0, 1, -1};
```

```
    int dy[4] = {1, -1, 0, 0};
```

```
    while(!rotten.empty()){
```

```
        int k = rotten.size();
```

```
        cnt += k;
```

```
        while(k--){
```

```
            int x = rotten.front().first, y = rotten.front().second;
```

```
            rotten.pop();
```

```
            for(int i = 0; i < 4; ++i){
```

```
                int nx = x + dx[i], ny = y + dy[i];
```

```
                if(nx < 0 || ny < 0 || nx >= m || ny >= n || grid[nx][ny] !=
```

```
1) continue;
```

```
                grid[nx][ny] = 2;
```

```
                rotten.push({nx, ny});
```

```
            }
```

```
        }
```

```
        if(!rotten.empty()) days++;
```

```
    }
```

```
    return tot == cnt ? days : -1;
```

```
}
```

```
int main()
```

```
{
```

```
    vector<vector<int>> v{ {2,1,1} , {1,1,0} , {0,1,1} } ;
```

```
    int rotting = orangesRotting(v);
```

```
    cout<<"Minimum Number of Minutes Required "<<rotting<<endl;
```

```
}
```

Flood Fill Algorithm - Graphs

Problem Statement: An image is represented by a 2-D array of integers, each integer representing the pixel value of the image. Given a coordinate (sr, sc) representing the starting pixel (row and column) of the flood fill, and a pixel value newColor, "flood fill" the image.

To perform a "flood fill", consider the starting pixel, plus any pixels connected 4-directionally to the starting pixel of the same colour as the starting pixel, plus any pixels connected 4-directionally to those pixels (also with the same colour as the starting pixel), and so on. Replace the colour of all of the aforementioned pixels with the newColor.

Pre-req: Connected Components, Graph traversal techniques

Example 1:

Input:

```
sr = 1, sc = 1, newColor = 2
#include<bits/stdc++.h>
using namespace std;

class Solution {
private:
    void dfs(int row, int col, vector<vector<int>>&ans,
            vector<vector<int>>& image, int newColor, int delRow[], int delCol[],
            int iniColor) {
        // color with new color
        ans[row][col] = newColor;
        int n = image.size();
        int m = image[0].size();
        // there are exactly 4 neighbours
        for(int i = 0; i < 4; i++) {
            int nrow = row + delRow[i];
            int ncol = col + delCol[i];
            // check for valid coordinate
            // then check for same initial color and unvisited pixel
            if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m &&
                image[nrow][ncol] == iniColor && ans[nrow][ncol] != newColor) {
                dfs(nrow, ncol, ans, image, newColor, delRow, delCol, iniColor);
            }
        }
    }
public:
    vector<vector<int>> floodFill(vector<vector<int>>& image,
        int sr, int sc, int newColor) {
        // get initial color
        int iniColor = image[sr][sc];
        vector<vector<int>> ans = image;
        // delta row and delta column for neighbours
        int delRow[] = {-1, 0, +1, 0};
        int delCol[] = {0, +1, 0, -1};
        dfs(sr, sc, ans, image, newColor, delRow, delCol, iniColor);
        return ans;
    }
};

int main(){
    vector<vector<int>>image{
        {1,1,1},
        {1,1,0},
        {1,0,1}
    };

    // sr = 1, sc = 1, newColor = 2
    Solution obj;
    vector<vector<int>> ans = obj.floodFill(image, 1, 1, 2);
    for(auto i: ans){
```

```

        for(auto j: i)
            cout << j << " ";
        cout << "\n";
    }

    return 0;
}

```

Detect Cycle in an Undirected Graph (using BFS)

Input:

V = 8, E = 6

Output: 1

```

#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    bool detect(int src, vector<int> adj[], int vis[]) {
        vis[src] = 1;
        // store <source node, parent node>
        queue<pair<int,int>> q;
        q.push({src, -1});
        // traverse until queue is not empty
        while(!q.empty()) {
            int node = q.front().first;
            int parent = q.front().second;
            q.pop();

            // go to all adjacent nodes
            for(auto adjacentNode: adj[node]) {
                // if adjacent node is unvisited
                if(!vis[adjacentNode]) {
                    vis[adjacentNode] = 1;
                    q.push({adjacentNode, node});
                }
                // if adjacent node is visited and is not it's own parent node
                else if(parent != adjacentNode) {
                    // yes it is a cycle
                    return true;
                }
            }
        }
        // there's no cycle
        return false;
    }
public:
    // Function to detect cycle in an undirected graph.
    bool isCycle(int V, vector<int> adj[]) {
        // initialise them as unvisited
        int vis[V] = {0};
        for(int i = 0; i < V; i++) {
            if(!vis[i]) {
                if(detect(i, adj, vis)) return true;
            }
        }
        return false;
    }
};

int main() {
    // V = 4, E = 2

```

```

vector<int> adj[4] = {{}, {2}, {1, 3}, {2}};
Solution obj;
bool ans = obj.isCycle(4, adj);
if (ans)
    cout << "1\n";
else
    cout << "0\n";
return 0;
}

```

Detect Cycle in an Undirected Graph (using DFS)

Example 2:

Input:

V = 8, E = 6

Output: Yes

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

class Solution {
private:
    bool dfs(int node, int parent, int vis[], vector<int> adj[]) {
        vis[node] = 1;
        // visit adjacent nodes
        for(auto adjacentNode: adj[node]) {
            // unvisited adjacent node
            if(!vis[adjacentNode]) {
                if(dfs(adjacentNode, node, vis, adj) == true)
                    return true;
            }
            // visited node but not a parent node
            else if(adjacentNode != parent) return true;
        }
        return false;
    }
public:
    // Function to detect cycle in an undirected graph.
    bool isCycle(int V, vector<int> adj[]) {
        int vis[V] = {0};
        // for graph with connected components
        for(int i = 0; i < V; i++) {
            if(!vis[i]) {
                if(dfs(i, -1, vis, adj) == true) return true;
            }
        }
        return false;
    }
};

```

```

int main() {

    // V = 4, E = 2
    vector<int> adj[4] = {{}, {2}, {1, 3}, {2}};
    Solution obj;
    bool ans = obj.isCycle(4, adj);
    if (ans)
        cout << "1\n";
    else
        cout << "0\n";
    return 0;
}

```

Distance of Nearest Cell having 1

Problem Statement: Given a binary grid of N*M. Find the distance of the nearest 1 in the grid for each cell.

The distance is calculated as $|i1 - i2| + |j1 - j2|$, where $i1, j1$ are the row number and column number of the current cell, and $i2, j2$ are the row number and column number of the nearest cell having value 1.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
//Function to find the distance of nearest 1 in the grid for each cell.
```

```
vector<vector<int>>nearest(vector<vector<int>>grid)
```

```
{
```

```
    int n = grid.size();
```

```
    int m = grid[0].size();
```

```
    // visited and distance matrix
```

```
    vector<vector<int>> vis(n, vector<int>(m, 0));
```

```
    vector<vector<int>> dist(n, vector<int>(m, 0));
```

```
    // <coordinates, steps>
```

```
    queue<pair<pair<int,int>, int>> q;
```

```
    // traverse the matrix
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < m; j++) {
```

```
            // start BFS if cell contains 1
```

```
            if(grid[i][j] == 1) {
```

```
                q.push({{i,j}, 0});
```

```
                vis[i][j] = 1;
```

```
            }
```

```
            else {
```

```
                // mark unvisited
```

```
                vis[i][j] = 0;
```

```
            }
```

```
        }
```

```
    }
```

```
    int delrow[] = {-1, 0, +1, 0};
```

```
    int delcol[] = {0, +1, 0, -1};
```

```
    // traverse till queue becomes empty
```

```
    while(!q.empty()) {
```

```
        int row = q.front().first.first;
```

```
        int col = q.front().first.second;
```

```
        int steps = q.front().second;
```

```
        q.pop();
```

```
        dist[row][col] = steps;
```

```
        // for all 4 neighbours
```

```
        for(int i = 0; i < 4; i++) {
```

```
            int nrow = row + delrow[i];
```

```
            int ncol = col + delcol[i];
```

```
            // check for valid unvisited cell
```

```
            if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
```

```
            && vis[nrow][ncol] == 0) {
```

```
                vis[nrow][ncol] = 1;
```

```
                q.push({{nrow, ncol}, steps+1});
```

```
            }
```

```
        }
```

```
    }
```

```
    // return distance matrix
```

```
    return dist;
```

```
}
```

```
};
```

```
int main(){
```

```
    vector<vector<int>>grid{
```

```
        {0,1,1,0},
```



```

        {1,1,0,0},
        {0,0,1,1}
    };

```

```

Solution obj;
vector<vector<int>> ans = obj.nearest(grid);

```

```

for(auto i: ans){
    for(auto j: i)
        cout << j << " ";
    cout << "\n";
}

```

```

return 0;

```

```

}

```

Surrounded Regions | Replace 0's with X's

Problem Statement: Given a matrix mat of size N x M where every element is either '0' or 'X'. Replace all '0' with 'X' that is surrounded by 'X'. An '0' (or a set of '0') is considered to be surrounded by 'X' if there are 'X' at locations just below, just above just left, and just right of it.

Examples:

Example 1:

Input: n = 5, m = 4

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution{

```

```

private:

```

```

    void dfs(int row, int col, vector<vector<int>> &vis,
    vector<vector<char>> &mat, int delrow[], int delcol[]) {
        vis[row][col] = 1;
        int n = mat.size();
        int m = mat[0].size();

        // check for top, right, bottom, left
        for(int i = 0; i < 4; i++) {
            int nrow = row + delrow[i];
            int ncol = col + delcol[i];
            // check for valid coordinates and unvisited 0s
            if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
            && !vis[nrow][ncol] && mat[nrow][ncol] == '0') {
                dfs(nrow, ncol, vis, mat, delrow, delcol);
            }
        }
    }
}

```

```

public:

```

```

    vector<vector<char>> fill(int n, int m,
    vector<vector<char>> mat)
    {
        int delrow[] = {-1, 0, +1, 0};
        int delcol[] = {0, 1, 0, -1};
        vector<vector<int>> vis(n, vector<int>(m,0));
        // traverse first row and last row
        for(int j = 0 ; j < m; j++) {
            // check for unvisited 0s in the boundary rows
            // first row
            if(!vis[0][j] && mat[0][j] == '0') {
                dfs(0, j, vis, mat, delrow, delcol);
            }

            // last row

```

```

        if(!vis[n-1][j] && mat[n-1][j] == '0') {
            dfs(n-1,j,vis,mat, delrow, delcol);
        }
    }

    for(int i = 0;i<n;i++) {
        // check for unvisited 0s in the boundary columns
        // first column
        if(!vis[i][0] && mat[i][0] == '0') {
            dfs(i, 0, vis, mat, delrow, delcol);
        }

        // last column
        if(!vis[i][m-1] && mat[i][m-1] == '0') {
            dfs(i, m-1, vis, mat, delrow, delcol);
        }
    }

    // if unvisited 0 then convert to X
    for(int i = 0;i<n;i++) {
        for(int j= 0 ;j<m;j++) {
            if(!vis[i][j] && mat[i][j] == '0')
                mat[i][j] = 'X';
        }
    }

    return mat;
}
};

```

```

int main(){

    vector<vector<char>> mat{
        {'X', 'X', 'X', 'X'},
        {'X', '0', 'X', 'X'},
        {'X', '0', '0', 'X'},
        {'X', '0', 'X', 'X'},
        {'X', 'X', '0', '0'}
    };

    Solution ob;
    // n = 5, m = 4
    vector<vector<char>> ans = ob.fill(5, 4, mat);
    for(int i = 0;i < 5;i++) {
        for(int j = 0;j < 4;j++) {
            cout<<ans[i][j]<<" ";
        }
        cout<<"\n";
    }
    return 0;
}

```

Number of Enclaves

Problem Statement: You are given an $N \times M$ binary matrix grid, where 0 represents a sea cell and 1 represents a land cell. A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid. Find the number of land cells in the grid for which we cannot walk off the boundary of the grid in any number of moves.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

class Solution {
public:
    int numberOfEnclaves(vector<vector<int>> &grid) {
        queue<pair<int,int>> q;
    }
}

```

```

int n = grid.size();
int m = grid[0].size();
int vis[n][m] = {0};
// traverse boundary elements
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        // first row, first col, last row, last col
        if(i == 0 || j == 0 || i == n-1 || j == m-1) {
            // if it is a land then store it in queue
            if(grid[i][j] == 1) {
                q.push({i, j});
                vis[i][j] = 1;
            }
        }
    }
}

int delrow[] = {-1, 0, +1, 0};
int delcol[] = {0, +1, +0, -1};

while(!q.empty()) {
    int row = q.front().first;
    int col = q.front().second;
    q.pop();

    // traverses all 4 directions
    for(int i = 0; i < 4; i++) {
        int nrow = row + delrow[i];
        int ncol = col + delcol[i];
        // check for valid coordinates and for land cell
        if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
        && vis[nrow][ncol] == 0 && grid[nrow][ncol] == 1) {
            q.push({nrow, ncol});
            vis[nrow][ncol] = 1;
        }
    }
}

int cnt = 0;
for(int i = 0; i < n; i++) {
    for(int j = 0; j < m; j++) {
        // check for unvisited land cell
        if(grid[i][j] == 1 & vis[i][j] == 0)
            cnt++;
    }
}
return cnt;
};

int main() {
    vector<vector<int>>> grid{
        {0, 0, 0, 0},
        {1, 0, 1, 0},
        {0, 1, 1, 0},
        {0, 0, 0, 0}};

    Solution obj;
    cout << obj.numberOfEnclaves(grid) << endl;
}

```

Mark as Completed

93

Given are the two distinct words startWord and targetWord, and a list denoting wordList of unique words of equal lengths. Find the length of the shortest transformation sequence from startWord to targetWord.

In this problem statement, we need to keep the following conditions in mind:

A word can only consist of lowercase characters.

Only one letter can be changed in each transformation.

Each transformed word must exist in the wordList including the targetWord.

startWord may or may not be part of the wordList

Note: If there's no possible way to transform the sequence from startWord to targetWord return 0.

Example 1:

Input:

```
wordList = {"des","der","dfr","dgt","dfs"}
```

```
startWord = "der", targetWord = "dfs"
```

Output:

3

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
    int wordLadderLength(string startWord, string targetWord,
                        vector<string> &wordList)
```

```
    {
```

```
        // Creating a queue ds of type {word, transitions to reach ~word~}.
        queue<pair<string, int>> q;
```

```
        // BFS traversal with pushing values in queue
```

```
        // when after a transformation, a word is found in wordList.
```

```
        q.push({startWord, 1});
```

```
        // Push all values of wordList into a set
```

```
        // to make deletion from it easier and in less time complexity.
```

```
        unordered_set<string> st(wordList.begin(), wordList.end());
```

```
        st.erase(startWord);
```

```
        while (!q.empty())
```

```
        {
```

```
            string word = q.front().first;
```

```
            int steps = q.front().second;
```

```
            q.pop();
```

```
            // we return the steps as soon as
```

```
            // the first occurrence of targetWord is found.
```

```
            if (word == targetWord)
```

```
                return steps;
```

```
            for (int i = 0; i < word.size(); i++)
```

```
            {
```

```
                // Now, replace each character of ~word~ with char
```

```
                // from a-z then check if ~word~ exists in wordList.
```

```
                char original = word[i];
```

```
                for (char ch = 'a'; ch <= 'z'; ch++)
```

```
                {
```



```

{
    // Push all values of wordList into a set
    // to make deletion from it easier and in less time complexity.
    unordered_set<string> st(wordList.begin(), wordList.end());

    // Creating a queue ds which stores the words in a sequence which is
    // required to reach the targetWord after successive transformations.
    queue<vector<string>> q;

    // BFS traversal with pushing the new formed sequence in queue
    // when after a transformation, a word is found in wordList.

    q.push({beginWord});

    // A vector defined to store the words being currently used
    // on a level during BFS.
    vector<string> usedOnLevel;
    usedOnLevel.push_back(beginWord);
    int level = 0;

    // A vector to store the resultant transformation sequence.
    vector<vector<string>> ans;
    while (!q.empty())
    {
        vector<string> vec = q.front();
        q.pop();

        // Now, erase all words that have been
        // used in the previous levels to transform
        if (vec.size() > level)
        {
            level++;
            for (auto it : usedOnLevel)
            {
                st.erase(it);
            }
        }

        string word = vec.back();

        // store the answers if the end word matches with targetWord.
        if (word == endWord)
        {
            // the first sequence where we reached end
            if (ans.size() == 0)
            {
                ans.push_back(vec);
            }
            else if (ans[0].size() == vec.size())
            {
                ans.push_back(vec);
            }
        }
        for (int i = 0; i < word.size(); i++)
        {
            // Now, replace each character of â~wordâ™™ with char
            // from a-z then check if â~wordâ™™ exists in wordList.
            char original = word[i];
            for (char c = 'a'; c <= 'z'; c++)
            {
                word[i] = c;
                if (st.count(word) > 0)
                {
                    // Check if the word is present in the wordList and

```

```

        // push the word along with the new sequence in the
queue.
        vec.push_back(word);
        q.push(vec);
        // mark as visited on the level
        usedOnLevel.push_back(word);
        vec.pop_back();
    }
    word[i] = original;
}
return ans;
}
};

```

```

// A comparator function to sort the answer.
bool comp(vector<string> a, vector<string> b)
{

```

```

    string x = "", y = "";
    for (string i : a)
        x += i;
    for (string i : b)
        y += i;

```

```

    return x < y;
}

```

```

int main()
{

```

```

    vector<string> wordList = {"des", "der", "dfr", "dgt", "dfs"};
    string startWord = "der", targetWord = "dfs";
    Solution obj;
    vector<vector<string>> ans = obj.findSequences(startWord, targetWord,
wordList);

```

```

    // If no transformation sequence is possible.

```

```

    if (ans.size() == 0)
        cout << -1 << endl;
    else
    {

```

```

        sort(ans.begin(), ans.end(), comp);
        for (int i = 0; i < ans.size(); i++)
        {
            for (int j = 0; j < ans[i].size(); j++)
            {
                cout << ans[i][j] << " ";
            }
            cout << endl;
        }
    }

```

```

    return 0;
}

```

Number of Islands

Problem Statement: Given a grid of size NxM (N is the number of rows and M is the number of columns in the grid) consisting of '0's (Water) and '1's (Land). Find the number of islands.

Note: An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically or diagonally i.e., in all 8 directions.

Pre-req: Connected Components, Graph traversal techniques

Examples:

Example 1:

Input:

Output: 3

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
private:
    void bfs(int row, int col, vector<vector<int>> &vis,
vector<vector<char>>&grid) {
        // mark it visited
        vis[row][col] = 1;
        queue<pair<int,int>> q;
        // push the node in queue
        q.push({row, col});
        int n = grid.size();
        int m = grid[0].size();

        // until the queue becomes empty
        while(!q.empty()) {
            int row = q.front().first;
            int col = q.front().second;
            q.pop();

            // traverse in the neighbours and mark them if its a land
            for(int delrow = -1; delrow<=1;delrow++) {
                for(int delcol = -1; delcol <= 1; delcol++) {
                    int nrow = row + delrow;
                    int ncol = col + delcol;
                    // neighbour row and column is valid, and is an unvisited land
                    if(nrow >= 0 && nrow < n && ncol >= 0 && ncol < m
&& grid[nrow][ncol] == '1' && !vis[nrow][ncol]) {
                        vis[nrow][ncol] = 1;
                        q.push({nrow, ncol});
                    }
                }
            }
        }
    }
public:
    // Function to find the number of islands.
    int numIslands(vector<vector<char>>& grid) {
        int n = grid.size();
        int m = grid[0].size();
        // create visited array and initialise to 0
        vector<vector<int>> vis(n, vector<int>(m, 0));
        int cnt = 0;
        for(int row = 0; row < n ; row++) {
            for(int col = 0; col < m ;col++) {
                // if not visited and is a land
                if(!vis[row][col] && grid[row][col] == '1') {
                    cnt++;
                    bfs(row, col, vis, grid);
                }
            }
        }
        return cnt;
    }
}
```



```
};

int main() {
    // n: row, m: column
    vector<vector<char>> grid
    {
        {'0', '1', '1', '1', '0', '0', '0'},
        {'0', '0', '1', '1', '0', '1', '0'}
    };

```

```
    Solution obj;
    cout << obj.numIslands(grid) << endl;

```

```
    return 0;
}

```

Bipartite Graph | DFS Implementation

Problem Statement: Given an adjacency list of a graph adj of V no. of vertices having 0 based index. Check whether the graph is bipartite or not.

If we are able to colour a graph with two colours such that no adjacent nodes have the same colour, it is called a bipartite graph.

Examples:

Example 1:

Input:

Output: 1

```
#include<bits/stdc++.h>
using namespace std;

```

```
class Solution {
private:
    bool dfs(int node, int col, int color[], vector<int> adj[]) {
        color[node] = col;

        // traverse adjacent nodes
        for(auto it : adj[node]) {
            // if uncoloured
            if(color[it] == -1) {
                if(dfs(it, !col, color, adj) == false) return false;
            }
            // if previously coloured and have the same colour
            else if(color[it] == col) {
                return false;
            }
        }

        return true;
    }
public:
    bool isBipartite(int V, vector<int>adj[]){
        int color[V];
        for(int i = 0;i<V;i++) color[i] = -1;

        // for connected components
        for(int i = 0;i<V;i++) {
            if(color[i] == -1) {
                if(dfs(i, 0, color, adj) == false)
                    return false;
            }
        }
    }
};

```

```

        }
    }
    return true;
}
};

void addEdge(vector<int> adj[], int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

```

```

int main(){

    // V = 4, E = 4
    vector<int>adj[4];

    addEdge(adj, 0, 2);
    addEdge(adj, 0, 3);
    addEdge(adj, 2, 3);
    addEdge(adj, 3, 1);

    Solution obj;
    bool ans = obj.isBipartite(4, adj);
    if(ans)cout << "1\n";
    else cout << "0\n";

    return 0;
}

```

Detect cycle in a directed graph (using DFS) : G 19

Problem Statement: Given a directed graph with V vertices and E edges, check whether it contains any cycle or not.

Example 1:

Input: N = 10, E = 11

Output: true

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
private:
    bool dfsCheck(int node, vector<int> adj[], int vis[], int pathVis[]) {
        vis[node] = 1;
        pathVis[node] = 1;

        // traverse for adjacent nodes
        for (auto it : adj[node]) {
            // when the node is not visited
            if (!vis[it]) {
                if (dfsCheck(it, adj, vis, pathVis) == true)
                    return true;
            }
            // if the node has been previously visited
            // but it has to be visited on the same path
            else if (pathVis[it]) {
                return true;
            }
        }

        pathVis[node] = 0;
    }
}

```

```

        return false;
    }
public:
    // Function to detect cycle in a directed graph.
    bool isCyclic(int V, vector<int> adj[]) {
        int vis[V] = {0};
        int pathVis[V] = {0};

        for (int i = 0; i < V; i++) {
            if (!vis[i]) {
                if (dfsCheck(i, adj, vis, pathVis) == true) return true;
            }
        }
        return false;
    }
};

```

```

int main() {
    // V = 11, E = 11;
    vector<int> adj[11] = {{}, {2}, {3}, {4, 7}, {5}, {6}, {}, {5}, {9}, {10},
{8}};
    int V = 11;
    Solution obj;
    bool ans = obj.isCyclic(V, adj);

    if (ans)
        cout << "True\n";
    else
        cout << "False\n";

    return 0;
}

```

Topological Sort Algorithm | DFS: G-21

Problem Statement: Given a Directed Acyclic Graph (DAG) with V vertices and E edges, Find any Topological Sorting of that Graph.

Note: In topological sorting, node u will always appear before node v if there is a directed edge from node u towards node v(u -> v).

Example 1:

Input: V = 6, E = 6

Output: 5, 4, 2, 3, 1, 0

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
private:
    void dfs(int node, int vis[], stack<int> &st,
            vector<int> adj[]) {
        vis[node] = 1;
        for (auto it : adj[node]) {
            if (!vis[it]) dfs(it, vis, st, adj);
        }
        st.push(node);
    }
public:
    //Function to return list containing vertices in Topological order.
    vector<int> topoSort(int V, vector<int> adj[])
    {

```

```

        int vis[V] = {0};
        stack<int> st;
        for (int i = 0; i < V; i++) {
            if (!vis[i]) {
                dfs(i, vis, st, adj);
            }
        }

        vector<int> ans;
        while (!st.empty()) {
            ans.push_back(st.top());
            st.pop();
        }
        return ans;
    }
};

```

```

int main() {
    //V = 6;
    vector<int> adj[6] = {{}, {}, {3}, {1}, {0, 1}, {0, 2}};
    int V = 6;
    Solution obj;
    vector<int> ans = obj.topoSort(V, adj);

    for (auto node : ans) {
        cout << node << " ";
    }
    cout << endl;

    return 0;
}

```

Kahn's Algorithm | Topological Sort Algorithm | BFS: G-22

Problem Statement: Given a Directed Acyclic Graph (DAG) with V vertices and E edges, Find any Topological Sorting of that Graph.

Note: In topological sorting, node u will always appear before node v if there is a directed edge from node u towards node v(u -> v).

Example 1:

Input Format: V = 6, E = 6

Result: 5, 4, 0, 2, 3, 1

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
public:
    //Function to return list containing vertices in Topological order.
    vector<int> topoSort(int V, vector<int> adj[])
    {
        int indegree[V] = {0};
        for (int i = 0; i < V; i++) {
            for (auto it : adj[i]) {
                indegree[it]++;
            }
        }

        queue<int> q;
        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {

```

```

        q.push(i);
    }
}
vector<int> topo;
while (!q.empty()) {
    int node = q.front();
    q.pop();
    topo.push_back(node);
    // node is in your topo sort
    // so please remove it from the indegree

    for (auto it : adj[node]) {
        indegree[it]--;
        if (indegree[it] == 0) q.push(it);
    }
}

return topo;
}
};

int main() {
    //V = 6;
    vector<int> adj[6] = {{}, {}, {3}, {1}, {0, 1}, {0, 2}};
    int V = 6;
    Solution obj;
    vector<int> ans = obj.topoSort(V, adj);

    for (auto node : ans) {
        cout << node << " ";
    }
    cout << endl;

    return 0;
}

```

Course Schedule I and II | Pre-requisite Tasks | Topological Sort: G-24

Problem Statement I (Course Schedule): There are a total of n tasks you have to pick, labeled from 0 to $n-1$. Some tasks may have prerequisite tasks, for example, to pick task 0 you have to first finish task 1 , which is expressed as a pair: $[0, 1]$

Given the total number of n tasks and a list of prerequisite pairs of size m . Find the order of tasks you should pick to finish all tasks.

Note: There may be multiple correct orders, you need to return one of them. If it is impossible to finish all tasks, return an empty array.

Problem Statement II (Pre-requisite Tasks): There are a total of N tasks, labeled from 0 to $N-1$. Some tasks may have prerequisites, for example, to do task 0 you have to first complete task 1 , which is expressed as a pair: $[0, 1]$

Given the total number of tasks N and a list of prerequisite pairs P , find if it is possible to finish all tasks.

Note: These two questions are linked. The second question asks if it is possible to finish all the tasks and the first question states to return the ordering of the tasks if it is possible to perform all the tasks, otherwise return an empty array.

Examples:

Example 1:

Input: N = 4, P = 3, array[] = {{1,0},{2,1},{3,2}}

Output: Yes

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
vector<int> findOrder(int V, int m, vector<vector<int>> prerequisites)
```

```
{
```

```
    vector<int> adj[V];
```

```
    for (auto it : prerequisites) {
```

```
        adj[it[1]].push_back(it[0]);
```

```
    }
```

```
    int indegree[V] = {0};
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (auto it : adj[i]) {
```

```
            indegree[it]++;
```

```
        }
```

```
    }
```

```
    queue<int> q;
```

```
    for (int i = 0; i < V; i++) {
```

```
        if (indegree[i] == 0) {
```

```
            q.push(i);
```

```
        }
```

```
    }
```

```
    vector<int> topo;
```

```
    while (!q.empty()) {
```

```
        int node = q.front();
```

```
        q.pop();
```

```
        topo.push_back(node);
```

```
        // node is in your topo sort
```

```
        // so please remove it from the indegree
```

```
        for (auto it : adj[node]) {
```

```
            indegree[it]--;
```

```
            if (indegree[it] == 0) q.push(it);
```

```
        }
```

```
    }
```

```
    if (topo.size() == V) return topo;
```

```
    return {};
```

```
}
```

```
};
```

```
int main() {
```

```
    int N = 4;
```

```
    int M = 3;
```

```
    vector<vector<int>> prerequisites(3);
```

```
    prerequisites[0].push_back(0);
```

```
    prerequisites[0].push_back(1);
```

```
    prerequisites[1].push_back(1);
```

```
    prerequisites[1].push_back(2);
```

```
    prerequisites[2].push_back(2);
```

```

prerequisites[2].push_back(3);

Solution obj;
vector<int> ans = obj.findOrder(N, M, prerequisites);

for (auto task : ans) {
    cout << task << " ";
}
cout << endl;
return 0;
}
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isPossible(int V, vector<pair<int, int> >& prerequisites) {
        vector<int> adj[V];
        for (auto it : prerequisites) {
            adj[it.first].push_back(it.second);
        }

        int indegree[V] = {0};
        for (int i = 0; i < V; i++) {
            for (auto it : adj[i]) {
                indegree[it]++;
            }
        }

        queue<int> q;
        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }
        vector<int> topo;
        while (!q.empty()) {
            int node = q.front();
            q.pop();
            topo.push_back(node);
            // node is in your topo sort
            // so please remove it from the indegree

            for (auto it : adj[node]) {
                indegree[it]--;
                if (indegree[it] == 0) q.push(it);
            }
        }

        if (topo.size() == V) return true;
        return false;
    }
};

int main() {
    vector<pair<int, int>> prerequisites;
    int N = 4;
    prerequisites.push_back({1, 0});
    prerequisites.push_back({2, 1});

```

```

prerequisites.push_back({3, 2});

Solution obj;
bool ans = obj.isPossible(N, prerequisites);

if (ans) cout << "YES";
else cout << "NO";
cout << endl;

return 0;
}

```

Find Eventual Safe States - BFS - Topological Sort: G-25

Problem Statement: A directed graph of V vertices and E edges is given in the form of an adjacency list `adj`. Each node of the graph is labeled with a distinct integer in the range 0 to $V - 1$. A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node. You have to return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Example 1:

Input Format: $N = 7, E = 7$

Result: {2 4 5 6}

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
public:
    vector<int> eventualSafeNodes(int V, vector<int> adj[]) {
        vector<int> adjRev[V];
        int indegree[V] = {0};
        for (int i = 0; i < V; i++) {
            // i -> it
            // it -> i
            for (auto it : adj[i]) {
                adjRev[it].push_back(i);
                indegree[i]++;
            }
        }
        queue<int> q;
        vector<int> safeNodes;
        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        while (!q.empty()) {
            int node = q.front();
            q.pop();
            safeNodes.push_back(node);
            for (auto it : adjRev[node]) {
                indegree[it]--;
                if (indegree[it] == 0) q.push(it);
            }
        }

        sort(safeNodes.begin(), safeNodes.end());
        return safeNodes;
    }
};

```



```

int main() {
    vector<int> adj[12] = {{1}, {2}, {3, 4}, {4, 5}, {6}, {6}, {7}, {}, {1,
9}, {10},
    {8}, {9}
    };
    int V = 12;
    Solution obj;
    vector<int> safeNodes = obj.eventualSafeNodes(V, adj);

    for (auto node : safeNodes) {
        cout << node << " ";
    }
    cout << endl;

    return 0;
}

```

Alien Dictionary - Topological Sort: G-26

Problem Statement: Given a sorted dictionary of an alien language having N words and k starting alphabets of a standard dictionary. Find the order of characters in the alien language.

Note: Many orders may be possible for a particular test case, thus you may return any valid order.

Examples:

Example 1:

Input: N = 5, K = 4

dict = {"baa", "abcd", "abca", "cab", "cad"}

Output: b d a c

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

class Solution {
    // works for multiple components
private:
    vector<int> topoSort(int V, vector<int> adj[])
    {
        int indegree[V] = {0};
        for (int i = 0; i < V; i++) {
            for (auto it : adj[i]) {
                indegree[it]++;
            }
        }

        queue<int> q;
        for (int i = 0; i < V; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }
        vector<int> topo;
        while (!q.empty()) {
            int node = q.front();
            q.pop();
            topo.push_back(node);
            // node is in your topo sort
            // so please remove it from the indegree

            for (auto it : adj[node]) {
                indegree[it]--;
            }
        }
    }
};

```

```

        if (indegree[it] == 0) q.push(it);
    }
}

return topo;
}

public:
    string findOrder(string dict[], int N, int K) {
        vector<int>adj[K];
        for (int i = 0; i < N - 1; i++) {
            string s1 = dict[i];
            string s2 = dict[i + 1];
            int len = min(s1.size(), s2.size());
            for (int ptr = 0; ptr < len; ptr++) {
                if (s1[ptr] != s2[ptr]) {
                    adj[s1[ptr] - 'a'].push_back(s2[ptr] - 'a');
                    break;
                }
            }
        }

        vector<int> topo = topoSort(K, adj);
        string ans = "";
        for (auto it : topo) {
            ans = ans + char(it + 'a');
        }
        return ans;
    }
};

int main() {
    int N = 5, K = 4;
    string dict[] = {"baa", "abcd", "abca", "cab", "cad"};
    Solution obj;
    string ans = obj.findOrder(dict, N, K);

    for (auto ch : ans)
        cout << ch << ' ';
    cout << endl;

    return 0;
}

```

Shortest Path in Undirected Graph with unit distance: G-28

Given an Undirected Graph having unit weight, find the shortest path from the source to all other nodes in this graph. In this problem statement, we have assumed the source vertex to be $\hat{0}$. If a vertex is unreachable from the source node, then return -1 for that vertex.

Example 1:

Input:

n = 9, m = 10

edges = [[0,1],[0,3],[3,4],[4,5],[5,6],[1,2],[2,6],[6,7],[7,8],[6,8]]

src=0

Output: 0 1 2 1 2 3 3 4 4

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```

vector<int> shortestPath(vector<vector<int>>& edges, int N,int M, int src){
//Create an adjacency list of size N for storing the undirected graph.
vector<int> adj[N];
for(auto it : edges) {
    adj[it[0]].push_back(it[1]);
    adj[it[1]].push_back(it[0]);
}

//A dist array of size N initialised with a large number to
//indicate that initially all the nodes are untraversed.

int dist[N];
for(int i = 0;i<N;i++) dist[i] = 1e9;
// BFS Implementation.
dist[src] = 0;
queue<int> q;
q.push(src);
while(!q.empty()) {
    int node = q.front();
    q.pop();
    for(auto it : adj[node]) {
        if(dist[node] + 1 < dist[it]) {
            dist[it] = 1 + dist[node];
            q.push(it);
        }
    }
}
// Updated shortest distances are stored in the resultant array
vector<int> ans(N, -1);
for(int i = 0;i<N;i++) {
    if(dist[i] != 1e9) {
        ans[i] = dist[i];
    }
}
return ans;
}
};

```

```

int main(){

int N=9, M=10;
vector<vector<int>> edges= {{0,1},{0,3},{3,4},{4,5},{5,6},{1,2},{2,6},{6,7},
{7,8},{6,8}};

```

```

Solution obj;
vector<int> ans = obj.shortestPath(edges,N,M,0);

```

```

for(int i=0;i<ans.size();i++){

    cout<<ans[i]<<" ";
}

```

```

return 0;
}

```

Shortest Path in Directed Acyclic Graph Topological Sort: G-27

Given a DAG, find the shortest path from the source to all other nodes in this DAG. In this problem statement, we have assumed the source vertex to be 0. You will be given the weighted edges of the graph.

Note: What is a DAG (Directed Acyclic Graph)?

A Directed Graph (containing one-sided edges) having no cycles is said to be a

Directed Acyclic Graph.

Examples:

Example 1:

Input: n = 6, m= 7

edges =[[0,1,2],[0,4,1],[4,5,4],[4,2,2],[1,2,3],[2,3,6],[5,3,1]]

Output: 0 2 3 6 1 5

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
private:
```

```
void topoSort(int node, vector < pair < int, int >> adj[],
```

```
int vis[], stack < int > & st) {
```

```
//This is the function to implement Topological sort.
```

```
vis[node] = 1;
```

```
for (auto it: adj[node]) {
```

```
int v = it.first;
```

```
if (!vis[v]) {
```

```
topoSort(v, adj, vis, st);
```

```
}
```

```
}
```

```
st.push(node);
```

```
}
```

```
public:
```

```
vector < int > shortestPath(int N, int M, vector < vector < int >> & edges)
```

```
{
```

```
//We create a graph first in the form of an adjacency list.
```

```
vector < pair < int, int >> adj[N];
```

```
for (int i = 0; i < M; i++) {
```

```
int u = edges[i][0];
```

```
int v = edges[i][1];
```

```
int wt = edges[i][2];
```

```
adj[u].push_back({v, wt});
```

```
}
```

```
// A visited array is created with initially
```

```
// all the nodes marked as unvisited (0).
```

```
int vis[N] = {
```

```
0
```

```
};
```

```
//Now, we perform topo sort using DFS technique
```

```
//and store the result in the stack st.
```

```
stack < int > st;
```

```
for (int i = 0; i < N; i++) {
```

```
if (!vis[i]) {
```

```
topoSort(i, adj, vis, st);
```

```
}
```

```
}
```

```
//Further, we declare a vector  $\hat{dist}$  in which we update the value of  
the nodes™
```

```
//distance from the source vertex after relaxation of a particular node.
```

```
vector < int > dist(N);
```

```
for (int i = 0; i < N; i++) {
```

```

        dist[i] = 1e9;
    }

    dist[0] = 0;
    while (!st.empty()) {
        int node = st.top();
        st.pop();

        for (auto it: adj[node]) {
            int v = it.first;
            int wt = it.second;

            if (dist[node] + wt < dist[v]) {
                dist[v] = wt + dist[node];
            }
        }
    }

    for (int i = 0; i < N; i++) {
        if (dist[i] == 1e9) dist[i] = -1;
    }
    return dist;
}

};

int main() {

    int N = 6, M = 7;

    vector<vector<int>> edges= {{0,1,2},{0,4,1},{4,5,4},{4,2,2},{1,2,3},{2,3,6},
{5,3,1}};
    Solution obj;
    vector < int > ans = obj.shortestPath(N, M, edges);

    for (int i = 0; i < ans.size(); i++) {

        cout << ans[i] << " ";
    }

    return 0;
}

```

Dijkstra's Algorithm - Using Set : G-33

Given a weighted, undirected, and connected graph of V vertices and an adjacency list adj where adj[i] is a list of lists containing two integers where the first integer of each list j denotes there is an edge between i and j, second integers corresponds to the weight of that edge. You are given the source vertex S and You have to Find the shortest distance of all the vertex from the source vertex S. You have to return a list of integers denoting the shortest distance between each node and Source vertex S.

Note: The Graph doesn't contain any negative weight cycle

Example 1:

Input:

V = 2
adj [] = {{1, 9}}, {{0, 9}}}
S = 0

Output:

0 9

```

#include<bits/stdc++.h>
using namespace std;

class Solution
{
public:
    //Function to find the shortest distance of all the vertices
    //from the source vertex S.
    vector<int> dijkstra(int V, vector<vector<int>> adj[], int S)
    {
        // Create a set ds for storing the nodes as a pair {dist,node}
        // where dist is the distance from source to the node.
        // set stores the nodes in ascending order of the distances
        set<pair<int,int>> st;

        // Initialising dist list with a large number to
        // indicate the nodes are unvisited initially.
        // This list contains distance from source to the nodes.
        vector<int> dist(V, 1e9);

        st.insert({0, S});

        // Source initialised with dist=0
        dist[S] = 0;

        // Now, erase the minimum distance node first from the set
        // and traverse for all its adjacent nodes.
        while(!st.empty()) {
            auto it = *(st.begin());
            int node = it.second;
            int dis = it.first;
            st.erase(it);

            // Check for all adjacent nodes of the erased
            // element whether the prev dist is larger than current or not.
            for(auto it : adj[node]) {
                int adjNode = it[0];
                int edgW = it[1];

                if(dis + edgW < dist[adjNode]) {
                    // erase if it was visited previously at
                    // a greater cost.
                    if(dist[adjNode] != 1e9)
                        st.erase({dist[adjNode], adjNode});

                    // If current distance is smaller,
                    // push it into the queue
                    dist[adjNode] = dis + edgW;
                    st.insert({dist[adjNode], adjNode});
                }
            }
        }
        // Return the list containing shortest distances
        // from source to all the nodes.
        return dist;
    }
};

int main()
{
    // Driver code.
    int V = 3, E = 3, S = 2;
    vector<vector<int>> adj[V];
    vector<vector<int>> edges;

```

```

vector<int> v1{1, 1}, v2{2, 6}, v3{2, 3}, v4{0, 1}, v5{1, 3}, v6{0, 6};
int i = 0;
adj[0].push_back(v1);
adj[0].push_back(v2);
adj[1].push_back(v3);
adj[1].push_back(v4);
adj[2].push_back(v5);
adj[2].push_back(v6);

```

```

Solution obj;
vector<int> res = obj.dijkstra(V, adj, S);

```

```

for (int i = 0; i < V; i++)
{
    cout << res[i] << " ";
}
cout << endl;
return 0;

```

```

}

```

Dijkstra's Algorithm - Using Priority Queue : G-32

Given a weighted, undirected, and connected graph of V vertices and an adjacency list adj where adj[i] is a list of lists containing two integers where the first integer of each list j denotes there is an edge between i and j, second integers corresponds to the weight of that edge. You are given the source vertex S and You have to Find the shortest distance of all the vertex from the source vertex S. You have to return a list of integers denoting the shortest distance between each node and the Source vertex S.

Note: The Graph doesn't contain any negative weight cycle.

Examples:

Example 1:

Input:

V = 2

adj [] = {{1, 9}}, {{0, 9}}

S = 0

Output:

0 9

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution
{

```

```

public:

```

```

    // Function to find the shortest distance of all the vertices
    // from the source vertex S.

```

```

    vector<int> dijkstra(int V, vector<vector<int>> adj[], int S)
    {

```

```

        // Create a priority queue for storing the nodes as a pair {dist,node}
        // where dist is the distance from source to the node.

```

```

        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;

```

```

        // Initialising distTo list with a large number to
        // indicate the nodes are unvisited initially.

```

```

// This list contains distance from source to the nodes.
vector<int> distTo(V, INT_MAX);

// Source initialised with dist=0.
distTo[S] = 0;
pq.push({0, S});

// Now, pop the minimum distance node first from the min-heap
// and traverse for all its adjacent nodes.
while (!pq.empty())
{
    int node = pq.top().second;
    int dis = pq.top().first;
    pq.pop();

    // Check for all adjacent nodes of the popped out
    // element whether the prev dist is larger than current or not.
    for (auto it : adj[node])
    {
        int v = it[0];
        int w = it[1];
        if (dis + w < distTo[v])
        {
            distTo[v] = dis + w;

            // If current distance is smaller,
            // push it into the queue.
            pq.push({dis + w, v});
        }
    }
}
// Return the list containing shortest distances
// from source to all the nodes.
return distTo;
}
};

```

```

int main()
{
    // Driver code.
    int V = 3, E = 3, S = 2;
    vector<vector<int>> adj[V];
    vector<vector<int>> edges;
    vector<int> v1{1, 1}, v2{2, 6}, v3{2, 3}, v4{0, 1}, v5{1, 3}, v6{0, 6};
    int i = 0;
    adj[0].push_back(v1);
    adj[0].push_back(v2);
    adj[1].push_back(v3);
    adj[1].push_back(v4);
    adj[2].push_back(v5);
    adj[2].push_back(v6);

    Solution obj;
    vector<int> res = obj.dijkstra(V, adj, S);

    for (int i = 0; i < V; i++)
    {
        cout << res[i] << " ";
    }
    cout << endl;
    return 0;
}

```

G-36: Shortest Distance in a Binary Maze

Given an $n * m$ matrix grid where each element can either be 0 or 1. You need to

find the shortest distance between a given source cell to a destination cell. The path can only be created out of a cell if its value is 1.

If the path is not possible between the source cell and the destination cell, then return -1.

Note: You can move into an adjacent cell if that adjacent cell is filled with element 1. Two cells are adjacent if they share a side. In other words, you can move in one of four directions, Up, Down, Left, and Right.

Examples:

Example 1:

Input:

```
grid[][] = {{1, 1, 1, 1},
             {1, 1, 0, 1},
             {1, 1, 1, 1},
             {1, 1, 0, 0},
             {1, 0, 0, 1}}
```

source = {0, 1}

destination = {2, 2}

Output:

3

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
    int shortestPath(vector<vector<int>> &grid, pair<int, int> source,
                    pair<int, int> destination)
```

```
    {
```

```
        // Edge Case: if the source is only the destination.
```

```
        if (source.first == destination.first &&
            source.second == destination.second)
            return 0;
```

```
        // Create a queue for storing cells with their distances from source
        // in the form {dist,{cell coordinates pair}}.
        queue<pair<int, pair<int, int>>> q;
        int n = grid.size();
        int m = grid[0].size();
```

```
        // Create a distance matrix with initially all the cells marked as
        // unvisited and the source cell as 0.
        vector<vector<int>> dist(n, vector<int>(m, 1e9));
        dist[source.first][source.second] = 0;
        q.push({0, {source.first, source.second}});
```

```
        // The following delta rows and delts columns array are created such
```

that

```
        // each index represents each adjacent node that a cell may have
        // in a direction.
        int dr[] = {-1, 0, 1, 0};
        int dc[] = {0, 1, 0, -1};
```

```
        // Iterate through the maze by popping the elements out of the queue
        // and pushing whenever a shorter distance to a cell is found.
```

```
        while (!q.empty())
```

```
        {
```

```
            auto it = q.front();
            q.pop();
            int dis = it.first;
```

```

        int r = it.second.first;
        int c = it.second.second;

        // Through this loop, we check the 4 direction adjacent nodes
        // for a shorter path to destination.
        for (int i = 0; i < 4; i++)
        {
            int newr = r + dr[i];
            int newc = c + dc[i];

            // Checking the validity of the cell and updating if dist is
shorter.
            if (newr >= 0 && newr < n && newc >= 0 && newc < m && grid[newr]
[newc]
            == 1 && dis + 1 < dist[newr][newc])
            {
                dist[newr][newc] = 1 + dis;

                // Return the distance until the point when
                // we encounter the destination cell.
                if (newr == destination.first &&
                    newc == destination.second)
                    return dis + 1;
                q.push({1 + dis, {newr, newc}});
            }
        }
        // If no path is found from source to destination.
        return -1;
    }
};

int main()
{
    // Driver Code.

    pair<int, int> source, destination;
    source.first = 0;
    source.second = 1;
    destination.first = 2;
    destination.second = 2;

    vector<vector<int>>> grid = {{1, 1, 1, 1},
                                {1, 1, 0, 1},
                                {1, 1, 1, 1},
                                {1, 1, 0, 0},
                                {1, 0, 0, 1}};

    Solution obj;

    int res = obj.shortestPath(grid, source, destination);

    cout << res;
    cout << endl;

    return 0;
}

```

G-37: Path With Minimum Effort

You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of the cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

Examples:

Example 1:

Input:

heights = [[1,2,2],[3,8,2],[5,3,5]]

Output:

2

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
    int MinimumEffort(vector<vector<int>> &heights)
    {
```

```
        // Create a priority queue containing pairs of cells
        // and their respective distance from the source cell in the
        // form {diff, {row of cell, col of cell}}.
        priority_queue<pair<int, pair<int, int>>,
                      vector<pair<int, pair<int, int>>>,
                      greater<pair<int, pair<int, int>>>>
```

```
        pq;
```

```
        int n = heights.size();
        int m = heights[0].size();
```

```
        // Create a distance matrix with initially all the cells marked as
        // unvisited and the dist for source cell (0,0) as 0.
        vector<vector<int>> dist(n, vector<int>(m, 1e9));
        dist[0][0] = 0;
        pq.push({0, {0, 0}});
```

that

```
        // The following delta rows and delts columns array are created such
```

```
        // each index represents each adjacent node that a cell may have
        // in a direction.
        int dr[] = {-1, 0, 1, 0};
        int dc[] = {0, 1, 0, -1};
```

```
        // Iterate through the matrix by popping the elements out of the queue
        // and pushing whenever a shorter distance to a cell is found.
```

```
        while (!pq.empty())
        {
```

```
            auto it = pq.top();
            pq.pop();
            int diff = it.first;
            int row = it.second.first;
            int col = it.second.second;
```

```
            // Check if we have reached the destination cell,
            // return the current value of difference (which will be min).
            if (row == n - 1 && col == m - 1)
                return diff;
```

```
            for (int i = 0; i < 4; i++)
            {
                // row - 1, col
                // row, col + 1
                // row + 1, col
```

```

        // row, col - 1
        int newr = row + dr[i];
        int newc = col + dc[i];

        // Checking validity of the cell.
        if (newr >= 0 && newc >= 0 && newr < n && newc < m)
        {
            // Effort can be calculated as the max value of differences
            // between the heights of the node and its adjacent nodes.
            int newEffort = max(abs(heights[row][col] - heights[newr]
[newc]), diff);

            // If the calculated effort is less than the prev value
            // we update as we need the min effort.
            if (newEffort < dist[newr][newc])
            {
                dist[newr][newc] = newEffort;
                pq.push({newEffort, {newr, newc}});
            }
        }
    }
    return 0; // if unreachable
}
};

```

```

int main()
{
    // Driver Code.

    vector<vector<int>> heights = {{1, 2, 2}, {3, 8, 2}, {5, 3, 5}};

    Solution obj;

    int ans = obj.MinimumEffort(heights);

    cout << ans;
    cout << endl;

    return 0;
}

```

G-38: Cheapest Flights Within K Stops

There are n cities and m edges connected by some number of flights. You are given an array of flights where $\text{flights}[i] = [\text{from}_i, \text{to}_i, \text{price}_i]$ indicates that there is a flight from city from_i to city to_i with cost price_i . You have also given three integers src , dst , and k , and return the cheapest price from src to dst with at most k stops. If there is no such route, return -1 .

Example 1:

Input:

$n = 4$

$\text{flights} = [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]]$

$\text{src} = 0$

$\text{dst} = 3$

$k = 1$

Output:

700

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution
```

```

{
public:
    int CheapestFlight(int n, vector<vector<int>> &flights,
                       int src, int dst, int K)
    {
        // Create the adjacency list to depict airports and flights in
        // the form of a graph.
        vector<pair<int, int>> adj[n];
        for (auto it : flights)
        {
            adj[it[0]].push_back({it[1], it[2]});
        }

        // Create a queue which stores the node and their distances from the
        // source in the form of {stops, {node, dist}} with a "stops"
indicating
        // the no. of nodes between src and current node.
        queue<pair<int, pair<int, int>>> q;

        q.push({0, {src, 0}});

        // Distance array to store the updated distances from the source.
        vector<int> dist(n, 1e9);
        dist[src] = 0;

        // Iterate through the graph using a queue like in Dijkstra with
        // popping out the element with min stops first.
        while (!q.empty())
        {
            auto it = q.front();
            q.pop();
            int stops = it.first;
            int node = it.second.first;
            int cost = it.second.second;

            // We stop the process as soon as the limit for the stops reaches.
            if (stops > K)
                continue;
            for (auto iter : adj[node])
            {
                int adjNode = iter.first;
                int edW = iter.second;

                // We only update the queue if the new calculated dist is
                // less than the prev and the stops are also within limits.
                if (cost + edW < dist[adjNode] && stops <= K)
                {
                    dist[adjNode] = cost + edW;
                    q.push({stops + 1, {adjNode, cost + edW}});
                }
            }
        }
        // If the destination node is unreachable return a "-1"
        // else return the calculated dist from src to dst.
        if (dist[dst] == 1e9)
            return -1;
        return dist[dst];
    }
};

int main()
{
    // Driver Code.
    int n = 4, src = 0, dst = 3, K = 1;

```

```

        vector<vector<int>> flights = {{0, 1, 100}, {1, 2, 100}, {2, 0, 100}, {1, 3,
600},
        {2, 3, 200}};

        Solution obj;

        int ans = obj.CheapestFlight(n, flights, src, dst, K);

        cout << ans;
        cout << endl;

        return 0;
}

```

G-40: Number of Ways to Arrive at Destination

You are in a city that consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array $\hat{\text{roads}}$ where $\text{roads}[i] = [u_i, v_i, \text{time}_i]$ means that there is a road between intersections u_i and v_i that takes time_i minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$.

Example 1:

Input:

$n=7, m=10$

edges= $[[0, 6, 7], [0, 1, 2], [1, 2, 3], [1, 3, 3], [6, 3, 3], [3, 5, 1], [6, 5, 1], [2, 5, 1], [0, 4, 5], [4, 6, 2]]$

Output:

4

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution
{

```

```

public:
    int countPaths(int n, vector<vector<int>> &roads)
    {
        // Creating an adjacency list for the given graph.
        vector<pair<int, int>> adj[n];
        for (auto it : roads)
        {
            adj[it[0]].push_back({it[1], it[2]});
            adj[it[1]].push_back({it[0], it[2]});
        }

        // Defining a priority queue (min heap).
        priority_queue<pair<int, int>,
                      vector<pair<int, int>>, greater<pair<int, int>>> pq;

        // Initializing the dist array and the ways array
        // along with their first indices.
        vector<int> dist(n, INT_MAX), ways(n, 0);
        dist[0] = 0;
        ways[0] = 1;
        pq.push({0, 0});
    }

```

```

// Define modulo value
int mod = (int)(1e9 + 7);

// Iterate through the graph with the help of priority queue
// just as we do in Dijkstra's Algorithm.
while (!pq.empty())
{
    int dis = pq.top().first;
    int node = pq.top().second;
    pq.pop();

    for (auto it : adj[node])
    {
        int adjNode = it.first;
        int edW = it.second;

        // This "if" condition signifies that this is the first
        // time we're coming with this short distance, so we push
        // in PQ and keep the no. of ways the same.
        if (dis + edW < dist[adjNode])
        {
            dist[adjNode] = dis + edW;
            pq.push({dis + edW, adjNode});
            ways[adjNode] = ways[node];
        }

        // If we again encounter a node with the same short distance
        // as before, we simply increment the no. of ways.
        else if (dis + edW == dist[adjNode])
        {
            ways[adjNode] = (ways[adjNode] + ways[node]) % mod;
        }
    }
}
// Finally, we return the no. of ways to reach
// (n-1)th node modulo 10^9+7.
return ways[n - 1] % mod;
}
};

int main()
{
    // Driver Code.
    int n = 7;

    vector<vector<int>> edges = {{0, 6, 7}, {0, 1, 2}, {1, 2, 3}, {1, 3, 3}, {6,
3, 3}, {3, 5, 1}, {6, 5, 1}, {2, 5, 1}, {0, 4, 5}, {4, 6, 2}};

    Solution obj;

    int ans = obj.countPaths(n, edges);

    cout << ans;
    cout << endl;

    return 0;
}

```

G-39: Minimum Multiplications to Reach End

Given start, end, and an array arr of n numbers. At each step, the start is multiplied by any number in the array and then a mod operation with 100000 is done to get the new start.

Your task is to find the minimum steps in which the end can be achieved starting

from the start. If it is not possible to reach the end, then return -1.

Example 2:

Input:

```
arr[] = {2, 5, 7}
```

```
start = 3
```

```
end = 30
```

Output:

```
2
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
    int minimumMultiplications(vector<int> &arr,  
                                int start, int end)
```

```
    {
```

```
        // Create a queue for storing the numbers as a result of multiplication  
        // of the numbers in the array and the start number.
```

```
        queue<pair<int, int>> q;
```

```
        q.push({start, 0});
```

```
        // Create a dist array to store the no. of multiplications to reach  
        // a particular number from the start number.
```

```
        vector<int> dist(1000000, 1e9);
```

```
        dist[start] = 0;
```

```
        int mod = 1000000;
```

```
        // Multiply the start no. with each of numbers in the arr
```

```
        // until we get the end no.
```

```
        while (!q.empty())
```

```
        {
```

```
            int node = q.front().first;
```

```
            int steps = q.front().second;
```

```
            q.pop();
```

```
            for (auto it : arr)
```

```
            {
```

```
                int num = (it * node) % mod;
```

```
                // If the no. of multiplications are less than before
```

```
                // in order to reach a number, we update the dist array.
```

```
                if (steps + 1 < dist[num])
```

```
                {
```

```
                    dist[num] = steps + 1;
```

```
                    // Whenever we reach the end number
```

```
                    // return the calculated steps
```

```
                    if (num == end)
```

```
                        return steps + 1;
```

```
                    q.push({num, steps + 1});
```

```
                }
```

```
            }
```

```
        }
```

```
        // If the end no. is unattainable.
```

```
        return -1;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    // Driver Code.
```



```

int start = 3, end = 30;

vector<int> arr = {2, 5, 7};

Solution obj;

int ans = obj.minimumMultiplications(arr, start, end);

cout << ans;
cout << endl;

return 0;
}

```

Bellman Ford Algorithm: G-41

Problem Statement: Given a weighted, directed and connected graph of V vertices and E edges, Find the shortest distance of all the vertices from the source vertex S.

Note: If the Graph contains a negative cycle then return an array consisting of only -1.

Example 1:

Input Format:

V = 6,

E = [[3, 2, 6], [5, 3, 1], [0, 1, 5], [1, 5, -3], [1, 2, -2], [3, 4, -2], [2, 4, 3]],

S = 0

Result: 0 5 3 3 1 2

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
public:
    /* Function to implement Bellman Ford
    * edges: vector of vectors which represents the graph
    * S: source vertex to start traversing graph with
    * V: number of vertices
    */
    vector<int> bellman_ford(int V, vector<vector<int>>& edges, int S) {
        vector<int> dist(V, 1e8);
        dist[S] = 0;
        for (int i = 0; i < V - 1; i++) {
            for (auto it : edges) {
                int u = it[0];
                int v = it[1];
                int wt = it[2];
                if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
                    dist[v] = dist[u] + wt;
                }
            }
        }
        // Nth relaxation to check negative cycle
        for (auto it : edges) {
            int u = it[0];
            int v = it[1];
            int wt = it[2];
            if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {
                return { -1 };
            }
        }
    }
}

```

```

        return dist;
    }
};

int main() {
    int V = 6;
    vector<vector<int>> edges(7, vector<int>(3));
    edges[0] = {3, 2, 6};
    edges[1] = {5, 3, 1};
    edges[2] = {0, 1, 5};
    edges[3] = {1, 5, -3};
    edges[4] = {1, 2, -2};
    edges[5] = {3, 4, -2};
    edges[6] = {2, 4, 3};

    int S = 0;
    Solution obj;
    vector<int> dist = obj.bellman_ford(V, edges, S);
    for (auto d : dist) {
        cout << d << " ";
    }
    cout << endl;

    return 0;
}

```

Floyd Warshall Algorithm: G-42

Problem Statement: The problem is to find the shortest distances between every pair of vertices in a given edge-weighted directed graph. The graph is represented as an adjacency matrix of size $n \times n$. $\text{Matrix}[i][j]$ denotes the weight of the edge from i to j . If $\text{Matrix}[i][j] = -1$, it means there is no edge from i to j .

Do it in place.

Example 1:

Input Format:

```

matrix[][] = { {0, 2, -1, -1},
               {1, 0, 3, -1}, {-1, -1, 0, -1}, {3, 5, 4, 0} }

```

Result:

```

0 2 5 -1
1 0 3 -1
-1 -1 0 -1
3 5 4 0

```

```

#include <bits/stdc++.h>
using namespace std;

```

```

class Solution {
public:
    void shortest_distance(vector<vector<int>>&matrix) {
        int n = matrix.size();
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == -1) {
                    matrix[i][j] = 1e9;
                }
                if (i == j) matrix[i][j] = 0;
            }
        }
    }
}

```

```

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    matrix[i][j] = min(matrix[i][j],
                                         matrix[i][k] + matrix[k][j]);
                }
            }
        }

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 1e9) {
                    matrix[i][j] = -1;
                }
            }
        }
    }
};

```

```

int main() {
    int V = 4;
    vector<vector<int>> matrix(V, vector<int>(V, -1));
    matrix[0][1] = 2;
    matrix[1][0] = 1;
    matrix[1][2] = 3;
    matrix[3][0] = 3;
    matrix[3][1] = 5;
    matrix[3][2] = 4;

    Solution obj;
    obj.shortest_distance(matrix);

    for (auto row : matrix) {
        for (auto cell : row) {
            cout << cell << " ";
        }
        cout << endl;
    }

    return 0;
}

```

Find the City With the Smallest Number of Neighbours at a Threshold Distance:
G-43

Problem Statement: There are n cities numbered from 0 to $n-1$. Given the array edges where edges[i] = [fromi, toi, weighti] represents a bidirectional and weighted edge between cities fromi and toi, and given the integer distance Threshold. You need to find out a city with the smallest number of cities that are reachable through some path and whose distance is at most Threshold Distance, If there are multiple such cities, our answer will be the city with the greatest number.

Note: that the distance of a path, connecting cities i and j are equal to the sum of the edges' weights along that path.

Example 1:

Input Format:

N=4, M=4,

```
edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]],
distanceThreshold = 4
```

Result: 3

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int findCity(int n, int m, vector<vector<int>>& edges,
                int distanceThreshold) {
        vector<vector<int>> dist(n, vector<int> (n, INT_MAX));
        for (auto it : edges) {
            dist[it[0]][it[1]] = it[2];
            dist[it[1]][it[0]] = it[2];
        }
        for (int i = 0; i < n; i++) dist[i][i] = 0;
        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] == INT_MAX || dist[k][j] ==
INT_MAX)
                        continue;
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);
                }
            }
        }

        int cntCity = n;
        int cityNo = -1;
        for (int city = 0; city < n; city++) {
            int cnt = 0;
            for (int adjCity = 0; adjCity < n; adjCity++) {
                if (dist[city][adjCity] <= distanceThreshold)
                    cnt++;
            }

            if (cnt <= cntCity) {
                cntCity = cnt;
                cityNo = city;
            }
        }
        return cityNo;
    }
};
```

```
int main() {
    int n = 4;
    int m = 4;
    vector<vector<int>> edges = {{0, 1, 3}, {1, 2, 1}, {1, 3, 4}, {2, 3, 1}};
    int distanceThreshold = 4;

    Solution obj;
    int cityNo = obj.findCity(n, m, edges, distanceThreshold);
    cout << "The answer is node: " << cityNo << endl;

    return 0;
}
Minimum Spanning Tree - Theory: G-44
```

Prim's Algorithm - Minimum Spanning Tree - C++ and Java: G-45

Problem Statement: Given a weighted, undirected, and connected graph of V vertices and E edges. The task is to find the sum of weights of the edges of the Minimum Spanning Tree.

(Sometimes it may be asked to find the MST as well, where in the MST the edge-informations will be stored in the form {u, v}(u = starting node, v = ending node).)

Example 1:

Input Format:

V = 5, edges = { {0, 1, 2}, {0, 3, 6}, {1, 2, 3}, {1, 3, 8}, {1, 4, 5}, {4, 2, 7} }

Result: 16

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution
```

```
{
```

```
public:
```

```
//Function to find sum of weights of edges of the Minimum Spanning Tree.
```

```
int spanningTree(int V, vector<vector<int>> adj[])
```

```
{
```

```
    priority_queue<pair<int, int> ,
```

```
                vector<pair<int, int> >, greater<pair<int, int>>> pq;
```

```
    vector<int> vis(V, 0);
```

```
    // {wt, node}
```

```
    pq.push({0, 0});
```

```
    int sum = 0;
```

```
    while (!pq.empty()) {
```

```
        auto it = pq.top();
```

```
        pq.pop();
```

```
        int node = it.second;
```

```
        int wt = it.first;
```

```
        if (vis[node] == 1) continue;
```

```
        // add it to the mst
```

```
        vis[node] = 1;
```

```
        sum += wt;
```

```
        for (auto it : adj[node]) {
```

```
            int adjNode = it[0];
```

```
            int edW = it[1];
```

```
            if (!vis[adjNode]) {
```

```
                pq.push({edW, adjNode});
```

```
            }
```

```
        }
```

```
    }
```

```
    return sum;
```

```
}
```

```
};
```

```
int main() {
```

```
    int V = 5;
```

```
    vector<vector<int>> edges = {{0, 1, 2}, {0, 2, 1}, {1, 2, 1}, {2, 3, 2},  
{3, 4, 1}, {4, 2, 2}};
```

```
    vector<vector<int>> adj[V];
```

```
    for (auto it : edges) {
```

```
        vector<int> tmp(2);
```

```
        tmp[0] = it[1];
```

```

        tmp[1] = it[2];
        adj[it[0]].push_back(tmp);

        tmp[0] = it[0];
        tmp[1] = it[2];
        adj[it[1]].push_back(tmp);
    }

    Solution obj;
    int sum = obj.spanningTree(V, adj);
    cout << "The sum of all the edge weights: " << sum << endl;

    return 0;
}
Disjoint Set | Union by Rank | Union by Size | Path Compression: G-46
#include <bits/stdc++.h>
using namespace std;
class DisjointSet {
    vector<int> rank, parent;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }
};
int main() {
    DisjointSet ds(7);
    ds.unionByRank(1, 2);
    ds.unionByRank(2, 3);
    ds.unionByRank(4, 5);
    ds.unionByRank(6, 7);
    ds.unionByRank(5, 6);
    // if 3 and 7 same or not
    if (ds.findUPar(3) == ds.findUPar(7)) {
        cout << "Same\n";
    }
    else cout << "Not same\n";

    ds.unionByRank(3, 7);
}

```

```

        if (ds.findUPar(3) == ds.findUPar(7)) {
            cout << "Same\n";
        }
        else cout << "Not same\n";
        return 0;
    }
}
Disjoint Set | Union by Rank | Union by Size | Path Compression: G-46
#include <bits/stdc++.h>
using namespace std;
class DisjointSet {
    vector<int> rank, parent;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }

        int findUPar(int node) {
            if (node == parent[node])
                return node;
            return parent[node] = findUPar(parent[node]);
        }

        void unionByRank(int u, int v) {
            int ulp_u = findUPar(u);
            int ulp_v = findUPar(v);
            if (ulp_u == ulp_v) return;
            if (rank[ulp_u] < rank[ulp_v]) {
                parent[ulp_u] = ulp_v;
            }
            else if (rank[ulp_v] < rank[ulp_u]) {
                parent[ulp_v] = ulp_u;
            }
            else {
                parent[ulp_v] = ulp_u;
                rank[ulp_u]++;
            }
        }
    };
};
int main() {
    DisjointSet ds(7);
    ds.unionByRank(1, 2);
    ds.unionByRank(2, 3);
    ds.unionByRank(4, 5);
    ds.unionByRank(6, 7);
    ds.unionByRank(5, 6);
    // if 3 and 7 same or not
    if (ds.findUPar(3) == ds.findUPar(7)) {
        cout << "Same\n";
    }
    else cout << "Not same\n";

    ds.unionByRank(3, 7);

    if (ds.findUPar(3) == ds.findUPar(7)) {
        cout << "Same\n";
    }
    else cout << "Not same\n";
    return 0;
}

```

Kruskal's Algorithm - Minimum Spanning Tree : G-47

Problem Statement: Given a weighted, undirected, and connected graph of V vertices and E edges. The task is to find the sum of weights of the edges of the Minimum Spanning Tree.

Example 1:

Input Format:

V = 5, edges = { {0, 1, 2}, {0, 3, 6}, {1, 2, 3}, {1, 3, 8}, {1, 4, 5}, {4, 2, 7}}

Result: 16

```
#include <bits/stdc++.h>
using namespace std;
```

```
class DisjointSet {
    vector<int> rank, parent, size;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        size.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
}
```



```

    }
};
class Solution
{
public:
    //Function to find sum of weights of edges of the Minimum Spanning Tree.
    int spanningTree(int V, vector<vector<int>> adj[])
    {
        // 1 - 2 wt = 5
        /// 1 - > (2, 5)
        // 2 -> (1, 5)

        // 5, 1, 2
        // 5, 2, 1
        vector<pair<int, pair<int, int>>> edges;
        for (int i = 0; i < V; i++) {
            for (auto it : adj[i]) {
                int adjNode = it[0];
                int wt = it[1];
                int node = i;

                edges.push_back({wt, {node, adjNode}});
            }
        }
        DisjointSet ds(V);
        sort(edges.begin(), edges.end());
        int mstWt = 0;
        for (auto it : edges) {
            int wt = it.first;
            int u = it.second.first;
            int v = it.second.second;

            if (ds.findUPar(u) != ds.findUPar(v)) {
                mstWt += wt;
                ds.unionBySize(u, v);
            }
        }

        return mstWt;
    }
};

int main() {
    int V = 5;
    vector<vector<int>> edges = {{0, 1, 2}, {0, 2, 1}, {1, 2, 1}, {2, 3, 2}, {3,
4, 1}, {4, 2, 2}};
    vector<vector<int>> adj[V];
    for (auto it : edges) {
        vector<int> tmp(2);
        tmp[0] = it[1];
        tmp[1] = it[2];
        adj[it[0]].push_back(tmp);

        tmp[0] = it[0];
        tmp[1] = it[2];
        adj[it[1]].push_back(tmp);
    }

    Solution obj;
    int mstWt = obj.spanningTree(V, adj);
    cout << "The sum of all the edge weights: " << mstWt << endl;
    return 0;
}

```

Number of Operations to Make Network Connected - DSU: G-49.

Problem Statement: You are given a graph with n vertices and m edges. You can remove one edge from anywhere and add that edge between any two vertices in one operation. Find the minimum number of operations that will be required to make the graph connected. If it is not possible to make the graph connected, return -1.

Pre-requisite: Disjoint Set data structure

Example 1:

Input Format: $N = 4$, $M = 3$, $\text{Edge[]} = [[0, 1], [0, 2], [1, 2]]$

Result: 1

```
#include <bits/stdc++.h>
using namespace std;
```

```
//User function Template for C++
```

```
class DisjointSet {
```

```
public:
```

```
    vector<int> rank, parent, size;
```

```
    DisjointSet(int n) {
```

```
        rank.resize(n + 1, 0);
```

```
        parent.resize(n + 1);
```

```
        size.resize(n + 1);
```

```
        for (int i = 0; i <= n; i++) {
```

```
            parent[i] = i;
```

```
            size[i] = 1;
```

```
        }
```

```
    }
```

```
    int findUPar(int node) {
```

```
        if (node == parent[node])
```

```
            return node;
```

```
        return parent[node] = findUPar(parent[node]);
```

```
    }
```

```
    void unionByRank(int u, int v) {
```

```
        int ulp_u = findUPar(u);
```

```
        int ulp_v = findUPar(v);
```

```
        if (ulp_u == ulp_v) return;
```

```
        if (rank[ulp_u] < rank[ulp_v]) {
```

```
            parent[ulp_u] = ulp_v;
```

```
        }
```

```
        else if (rank[ulp_v] < rank[ulp_u]) {
```

```
            parent[ulp_v] = ulp_u;
```

```
        }
```

```
        else {
```

```
            parent[ulp_v] = ulp_u;
```

```
            rank[ulp_u]++;
```

```
        }
```

```
    }
```

```
    void unionBySize(int u, int v) {
```

```
        int ulp_u = findUPar(u);
```

```
        int ulp_v = findUPar(v);
```

```
        if (ulp_u == ulp_v) return;
```

```
        if (size[ulp_u] < size[ulp_v]) {
```

```
            parent[ulp_u] = ulp_v;
```

```
            size[ulp_v] += size[ulp_u];
```

```
        }
```

```
        else {
```

```
            parent[ulp_v] = ulp_u;
```

```

        size[ulp_u] += size[ulp_v];
    }
};

class Solution {
public:
    int Solve(int n, vector<vector<int>>& edge) {
        DisjointSet ds(n);
        int cntExtras = 0;
        for (auto it : edge) {
            int u = it[0];
            int v = it[1];
            if (ds.findUPar(u) == ds.findUPar(v)) {
                cntExtras++;
            }
            else {
                ds.unionBySize(u, v);
            }
        }
        int cntC = 0;
        for (int i = 0; i < n; i++) {
            if (ds.parent[i] == i) cntC++;
        }
        int ans = cntC - 1;
        if (cntExtras >= ans) return ans;
        return -1;
    }
};

int main() {

    int V = 9;
    vector<vector<int>> edge = {{0, 1}, {0, 2}, {0, 3}, {1, 2}, {2, 3}, {4, 5},
{5, 6}, {7, 8}};

    Solution obj;
    int ans = obj.Solve(V, edge);
    cout << "The number of operations needed: " << ans << endl;
    return 0;
}

```

Most Stones Removed with Same Row or Column - DSU: G-53

Problem Statement: There are n stones at some integer coordinate points on a 2D plane. Each coordinate point may have at most one stone.

You need to remove some stones.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array of stones of length n where stones[i] = [xi, yi] represents the location of the ith stone, return the maximum possible number of stones that you can remove.

Pre-requisite: Disjoint Set data structure

Example 1:

Input Format: n=6 stones = [[0, 0],[0, 1], [1, 0],[1, 2],[2, 1],[2, 2]]

Result: 5

```

#include <bits/stdc++.h>
using namespace std;

```

```

class DisjointSet {
    vector<int> rank, parent, size;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        size.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
};

class Solution {
public:
    int maxRemove(vector<vector<int>>& stones, int n) {
        int maxRow = 0;
        int maxCol = 0;
        for (auto it : stones) {
            maxRow = max(maxRow, it[0]);
            maxCol = max(maxCol, it[1]);
        }
        DisjointSet ds(maxRow + maxCol + 1);
        unordered_map<int, int> stoneNodes;
        for (auto it : stones) {
            int nodeRow = it[0];
            int nodeCol = it[1] + maxRow + 1;

```

```

        ds.unionBySize(nodeRow, nodeCol);
        stoneNodes[nodeRow] = 1;
        stoneNodes[nodeCol] = 1;
    }

    int cnt = 0;
    for (auto it : stoneNodes) {
        if (ds.findUPar(it.first) == it.first) {
            cnt++;
        }
    }
    return n - cnt;
}
};

int main() {
    int n = 6;
    vector<vector<int>> stones = {
        {0, 0}, {0, 2},
        {1, 3}, {3, 1},
        {3, 2}, {4, 3}
    };

    Solution obj;
    int ans = obj.maxRemove(stones, n);
    cout << "The maximum number of stones we can remove is: " << ans << endl;
    return 0;
}

```

Accounts Merge - DSU: G-50

Problem Statement: Given a list of accounts where each element account [i] is a list of strings, where the first element account [i][0] is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order.

Note: Accounts themselves can be returned in any order.

Pre-requisite: Disjoint Set data structure

Examples:

Example 1:

Input: N = 4

accounts [] =

```

[["John", "johnsmith@mail.com", "john_newyork@mail.com"],
["John", "johnsmith@mail.com", "john00@mail.com"],
["Mary", "mary@mail.com"],
["John", "johnnybravo@mail.com"]]

```

```

Output: [["John", "john00@mail.com", "john_newyork@mail.com",
"johnsmith@mail.com"],
["Mary", "mary@mail.com"],
["John", "johnnybravo@mail.com"]]

```

```

#include <bits/stdc++.h>

```

```

using namespace std;

```

```

//User function Template for C++
class DisjointSet {
    vector<int> rank, parent, size;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        size.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
};

class Solution {
public:
    vector<vector<string>> accountsMerge(vector<vector<string>> &details) {
        int n = details.size();
        DisjointSet ds(n);
        sort(details.begin(), details.end());
        unordered_map<string, int> mapMailNode;
        for (int i = 0; i < n; i++) {
            for (int j = 1; j < details[i].size(); j++) {
                string mail = details[i][j];
                if (mapMailNode.find(mail) == mapMailNode.end()) {
                    mapMailNode[mail] = i;
                }
            }
        }
    }
};

```

```

        else {
            ds.unionBySize(i, mapMailNode[mail]);
        }
    }
}

vector<string> mergedMail[n];
for (auto it : mapMailNode) {
    string mail = it.first;
    int node = ds.findUPar(it.second);
    mergedMail[node].push_back(mail);
}

vector<vector<string>> ans;

for (int i = 0; i < n; i++) {
    if (mergedMail[i].size() == 0) continue;
    sort(mergedMail[i].begin(), mergedMail[i].end());
    vector<string> temp;
    temp.push_back(details[i][0]);
    for (auto it : mergedMail[i]) {
        temp.push_back(it);
    }
    ans.push_back(temp);
}
sort(ans.begin(), ans.end());
return ans;
}
};

```

```

int main() {

    vector<vector<string>> accounts = {{"John", "j1@com", "j2@com", "j3@com"},
        {"John", "j4@com"},
        {"Raj", "r1@com", "r2@com"},
        {"John", "j1@com", "j5@com"},
        {"Raj", "r2@com", "r3@com"},
        {"Mary", "m1@com"}
    };
}

```

```

Solution obj;
vector<vector<string>> ans = obj.accountsMerge(accounts);
for (auto acc : ans) {
    cout << acc[0] << ":";
    int size = acc.size();
    for (int i = 1; i < size; i++) {
        cout << acc[i] << " ";
    }
    cout << endl;
}
return 0;
}

```

Number of Islands - II - Online Queries - DSU: G-51

Problem Statement: You are given an n, m which means the row and column of the 2D matrix, and an array of size k denoting the number of operations. Matrix elements are 0 if there is water or 1 if there is land. Originally, the 2D matrix is all 0 which means there is no land in the matrix. The array has k operator(s) and each operator has two integers $A[i][0]$, $A[i][1]$ means that you can change the cell matrix[$A[i][0]$][$A[i][1]$] from sea to island. Return how many islands are there in the matrix after each operation. You need to return an array of size k .

Note: An island means a group of 1s such that they share a common side.

Pre-requisite: Disjoint Set data structure

Example 1:

Input Format: $n = 4$ $m = 5$ $k = 4$ $A = \{\{1,1\},\{0,1\},\{3,3\},\{3,4\}\}$ Output: 1 1 2 2

Explanation: The following illustration is the representation of the operation:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// User function Template for C++
```

```
class DisjointSet {
    vector<int> rank, parent, size;
public:
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        size.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
            size[ulp_u] += size[ulp_v];
        }
    }
};

class Solution {
private:
    bool isValid(int adjr, int adjc, int n, int m) {
```



```

        return adjr >= 0 && adjr < n && adjc >= 0 && adjc < m;
    }
public:
    vector<int> numOfIslands(int n, int m,
                           vector<vector<int>> &operators) {
        DisjointSet ds(n * m);
        int vis[n][m];
        memset(vis, 0, sizeof vis);
        int cnt = 0;
        vector<int> ans;
        for (auto it : operators) {
            int row = it[0];
            int col = it[1];
            if (vis[row][col] == 1) {
                ans.push_back(cnt);
                continue;
            }
            vis[row][col] = 1;
            cnt++;
            // row - 1, col
            // row , col + 1
            // row + 1, col
            // row, col - 1;
            int dr[] = { -1, 0, 1, 0};
            int dc[] = {0, 1, 0, -1};
            for (int ind = 0; ind < 4; ind++) {
                int adjr = row + dr[ind];
                int adjc = col + dc[ind];
                if (isValid(adjr, adjc, n, m)) {
                    if (vis[adjr][adjc] == 1) {
                        int nodeNo = row * m + col;
                        int adjNodeNo = adjr * m + adjc;
                        if (ds.findUPar(nodeNo) != ds.findUPar(adjNodeNo)) {
                            cnt--;
                            ds.unionBySize(nodeNo, adjNodeNo);
                        }
                    }
                }
            }
            ans.push_back(cnt);
        }
        return ans;
    }
};

```

```

int main() {
    int n = 4, m = 5;
    vector<vector<int>> operators = {{0, 0}, {0, 0}, {1, 1}, {1, 0}, {0, 1},
                                     {0, 3}, {1, 3}, {0, 4}, {3, 2}, {2, 2}, {1, 2}, {0, 2}};

    Solution obj;
    vector<int> ans = obj.numOfIslands(n, m, operators);
    for (auto res : ans) {
        cout << res << " ";
    }
    cout << endl;
    return 0;
}

```

Making a Large Island - DSU: G-52

Problem Statement: You are given an $n \times n$ binary grid. A grid is said to be binary if every value in the grid is either 1 or 0. You can change at most one cell in the grid from 0 to 1. You need to find the largest group of connected 1's. Two cells are said to be connected if both are adjacent to each other and both have the same value.

Pre-requisite: Disjoint Set data structure

Example 1:

Input Format: The following grid is given:

Result: 20

```
#include <bits/stdc++.h>
using namespace std;
```

```
// User function Template for C++
class DisjointSet {
```

```
public:
```

```
    vector<int> rank, parent, size;
```

```
    DisjointSet(int n) {
        rank.resize(n + 1, 0);
        parent.resize(n + 1);
        size.resize(n + 1);
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
```

```
    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }
```

```
    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if (rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }
```

```
    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if (ulp_u == ulp_v) return;
        if (size[ulp_u] < size[ulp_v]) {
            parent[ulp_u] = ulp_v;
            size[ulp_v] += size[ulp_u];
        }
        else {
            parent[ulp_v] = ulp_u;
```

```

        size[ulp_u] += size[ulp_v];
    }
}
};
class Solution {
private:
    bool isValid(int newr, int newc, int n) {
        return newr >= 0 && newr < n && newc >= 0 && newc < n;
    }
public:
    int MaxConnection(vector<vector<int>>& grid) {
        int n = grid.size();
        DisjointSet ds(n * n);
        // step - 1
        for (int row = 0; row < n ; row++) {
            for (int col = 0; col < n ; col++) {
                if (grid[row][col] == 0) continue;
                int dr[] = { -1, 0, 1, 0};
                int dc[] = {0, -1, 0, 1};
                for (int ind = 0; ind < 4; ind++) {
                    int newr = row + dr[ind];
                    int newc = col + dc[ind];
                    if (isValid(newr, newc, n) && grid[newr][newc] == 1) {
                        int nodeNo = row * n + col;
                        int adjNodeNo = newr * n + newc;
                        ds.unionBySize(nodeNo, adjNodeNo);
                    }
                }
            }
        }
        // step 2
        int mx = 0;
        for (int row = 0; row < n; row++) {
            for (int col = 0; col < n; col++) {
                if (grid[row][col] == 1) continue;
                int dr[] = { -1, 0, 1, 0};
                int dc[] = {0, -1, 0, 1};
                set<int> components;
                for (int ind = 0; ind < 4; ind++) {
                    int newr = row + dr[ind];
                    int newc = col + dc[ind];
                    if (isValid(newr, newc, n)) {
                        if (grid[newr][newc] == 1) {
                            components.insert(ds.findUPar(newr * n + newc));
                        }
                    }
                }
                int sizeTotal = 0;
                for (auto it : components) {
                    sizeTotal += ds.size[it];
                }
                mx = max(mx, sizeTotal + 1);
            }
        }
        for (int cellNo = 0; cellNo < n * n; cellNo++) {
            mx = max(mx, ds.size[ds.findUPar(cellNo)]);
        }
        return mx;
    }
};

```

```

int main() {

```

```

vector<vector<int>> grid = {
    {1, 1, 0, 1, 1, 0}, {1, 1, 0, 1, 1, 0},
    {1, 1, 0, 1, 1, 0}, {0, 0, 1, 0, 0, 0},
    {0, 0, 1, 1, 1, 0}, {0, 0, 1, 1, 1, 0}
};

Solution obj;
int ans = obj.MaxConnection(grid);
cout << "The largest group of connected 1s is of size: " << ans << endl;
return 0;
}

```

Bridges in Graph - Using Tarjan's Algorithm of time in and low time: G-55

Problem Statement: There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other servers.

Return all critical connections in the network in any order.

Note: Here servers mean the nodes of the graph. The problem statement is taken from leetcode.

Pre-requisite: DFS algorithm

Example 1:

Input Format: $N = 4$, $\text{connections} = [[0,1],[1,2],[2,0],[1,3]]$

Result: $[[1, 3]]$

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
private:
```

```
    int timer = 1;
```

```
    void dfs(int node, int parent, vector<int> &vis,
```

```
            vector<int> adj[], int tin[], int low[], vector<vector<int>>
```

```
&bridges) {
```

```
    vis[node] = 1;
```

```
    tin[node] = low[node] = timer;
```

```
    timer++;
```

```
    for (auto it : adj[node]) {
```

```
        if (it == parent) continue;
```

```
        if (vis[it] == 0) {
```

```
            dfs(it, node, vis, adj, tin, low, bridges);
```

```
            low[node] = min(low[it], low[node]);
```

```
            // node --- it
```

```
            if (low[it] > tin[node]) {
```

```
                bridges.push_back({it, node});
```

```
            }
```

```
        }
```

```
    } else {
```

```
        low[node] = min(low[node], low[it]);
```

```
    }
```

```
    }
```

```
}
```

```
public:
```

```
    vector<vector<int>> criticalConnections(int n,
```

```
    vector<vector<int>>& connections) {
```

```
        vector<int> adj[n];
```

```

        for (auto it : connections) {
            int u = it[0], v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
        vector<int> vis(n, 0);
        int tin[n];
        int low[n];
        vector<vector<int>> bridges;
        dfs(0, -1, vis, adj, tin, low, bridges);
        return bridges;
    }
};

int main() {
    int n = 4;
    vector<vector<int>> connections = {
        {0, 1}, {1, 2},
        {2, 0}, {1, 3}
    };

    Solution obj;
    vector<vector<int>> bridges = obj.criticalConnections(n, connections);
    for (auto it : bridges) {
        cout << "[" << it[0] << ", " << it[1] << "]" << " ";
    }
    cout << endl;
    return 0;
}

```

Articulation Point in Graph: G-56

Problem Statement: Given an undirected connected graph with V vertices and adjacency list adj. You are required to find all the vertices removing which (and edges through it) disconnect the graph into 2 or more components.

Note: Indexing is zero-based i.e nodes numbering from (0 to V-1). There might be loops present in the graph.

Pre-requisite: Bridges in Graph problem & DFS algorithm.

Example 1:

Input Format:

Result: {0, 2}

```

#include <bits/stdc++.h>
using namespace std;

```

```

//User function Template for C++

```

```

class Solution {
private:
    int timer = 1;
    void dfs(int node, int parent, vector<int> &vis, int tin[], int low[],
            vector<int> &mark, vector<int> adj[]) {
        vis[node] = 1;
        tin[node] = low[node] = timer;
        timer++;
        int child = 0;
        for (auto it : adj[node]) {
            if (it == parent) continue;
            if (!vis[it]) {
                dfs(it, node, vis, tin, low, mark, adj);
            }
        }
    }
};

```

```

        low[node] = min(low[node], low[it]);
        if (low[it] >= tin[node] && parent != -1) {
            mark[node] = 1;
        }
        child++;
    }
    else {
        low[node] = min(low[node], tin[it]);
    }
}
if (child > 1 && parent == -1) {
    mark[node] = 1;
}
}
public:
    vector<int> articulationPoints(int n, vector<int>adj[]) {
        vector<int> vis(n, 0);
        int tin[n];
        int low[n];
        vector<int> mark(n, 0);
        for (int i = 0; i < n; i++) {
            if (!vis[i]) {
                dfs(i, -1, vis, tin, low, mark, adj);
            }
        }
        vector<int> ans;
        for (int i = 0; i < n; i++) {
            if (mark[i] == 1) {
                ans.push_back(i);
            }
        }
        if (ans.size() == 0) return { -1};
        return ans;
    }
};

int main() {

    int n = 5;
    vector<vector<int>> edges = {
        {0, 1}, {1, 4},
        {2, 4}, {2, 3}, {3, 4}
    };

    vector<int> adj[n];
    for (auto it : edges) {
        int u = it[0], v = it[1];
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    Solution obj;
    vector<int> nodes = obj.articulationPoints(n, adj);
    for (auto node : nodes) {
        cout << node << " ";
    }
    cout << endl;
    return 0;
}

```

Strongly Connected Components - Kosaraju's Algorithm: G-54

Problem Statement: Given a Directed Graph with V vertices (Numbered from 0 to V-1) and E edges, Find the number of strongly connected components in the graph.

Pre-requisite: DFS algorithm

Example 1:

Input Format:

Result: 3

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution
{
private:
    void dfs(int node, vector<int> &vis, vector<int> adj[],
             stack<int> &st) {
        vis[node] = 1;
        for (auto it : adj[node]) {
            if (!vis[it]) {
                dfs(it, vis, adj, st);
            }
        }
        st.push(node);
    }
private:
    void dfs3(int node, vector<int> &vis, vector<int> adjT[]) {
        vis[node] = 1;
        for (auto it : adjT[node]) {
            if (!vis[it]) {
                dfs3(it, vis, adjT);
            }
        }
    }
public:
    //Function to find number of strongly connected components in the graph.
    int kosaraju(int V, vector<int> adj[])
    {
        vector<int> vis(V, 0);
        stack<int> st;
        for (int i = 0; i < V; i++) {
            if (!vis[i]) {
                dfs(i, vis, adj, st);
            }
        }

        vector<int> adjT[V];
        for (int i = 0; i < V; i++) {
            vis[i] = 0;
            for (auto it : adj[i]) {
                // i -> it
                // it -> i
                adjT[it].push_back(i);
            }
        }
        int scc = 0;
        while (!st.empty()) {
            int node = st.top();
            st.pop();
            if (!vis[node]) {
                scc++;
                dfs3(node, vis, adjT);
            }
        }
    }
}
```

```

        return scc;
    }
};

int main() {
    int n = 5;
    int edges[5][2] = {
        {1, 0}, {0, 2},
        {2, 1}, {0, 3},
        {3, 4}
    };
    vector<int> adj[n];
    for (int i = 0; i < n; i++) {
        adj[edges[i][0]].push_back(edges[i][1]);
    }
    Solution obj;
    int ans = obj.kosaraju(n, adj);
    cout << "The number of strongly connected components is: " << ans << endl;
    return 0;
}

```